Creación de secuencias de comandos y estructuras de control

Los sistemas gestores de bases de datos permiten crear programas almacenados que ejecutan varias instrucciones SQL de forma organizada. Mediante estos programas es posible automatizar operaciones administrativas, aplicar condiciones, repetir instrucciones y controlar el comportamiento de las operaciones.

En este bloque se estudiarán las variables, los procedimientos almacenados y las estructuras de control de flujo disponibles en MariaDB. También se utilizarán parámetros para comunicar información entre los procedimientos y las aplicaciones que los ejecutan.

Objetivos del bloque

- Comprender el funcionamiento de las variables SQL.
- Crear y ejecutar procedimientos almacenados.
- Utilizar parámetros de entrada y salida.
- Aplicar estructuras condicionales y repetitivas.
- Automatizar operaciones DDL y DML.
- Controlar errores durante la ejecución de procedimientos.
- Proteger la integridad y consistencia de los datos.

1. Variables y secuencias de comandos SQL

1.1. ¿Qué es una variable?

Una variable es un espacio de almacenamiento temporal que permite guardar un valor para utilizarlo posteriormente durante la ejecución de un programa.

En los procedimientos almacenados de MariaDB, las variables permiten almacenar números, textos, fechas y otros valores que pueden utilizarse en consultas y operaciones SQL.

Por ejemplo, un procedimiento puede almacenar el número de alumnos de un curso, calcular un importe o guardar el resultado de una consulta para utilizarlo en una operación posterior.

1.2. Declaración de variables con DECLARE

Dentro de un procedimiento almacenado, las variables locales se declaran utilizando la instrucción `DECLARE`.

La declaración debe realizarse al principio del bloque `BEGIN ... END`, antes de las instrucciones ejecutables. Primero se declaran las variables y después se escriben las operaciones que las utilizan.

Sintaxis general:

```
DECLARE nombre_variable tipo_dato;
```

Algunos tipos de datos habituales son:

Tipo	Descripción	Ejemplo
INT	Números enteros.	25
DECIMAL(10,2)	Números decimales con precisión definida.	1250.75
VARCHAR(100)	Cadenas de texto de longitud variable.	'Administración'
DATE	Fechas.	'2026-11-12'
DATETIME	Fecha y hora.	'2026-11-12 10:30:00'
BOOLEAN	Valores lógicos, representados internamente como 0 y 1.	TRUE

Ejemplo: declaración de variables

```
DELIMITER //

CREATE PROCEDURE ejemplo_variables()
BEGIN
    DECLARE cantidad INT;
    DECLARE nombre VARCHAR(100);
    DECLARE precio DECIMAL(10,2);
    DECLARE fecha DATE;
END //

DELIMITER ;
```

El procedimiento declara cuatro variables locales. Todavía no realiza ninguna operación con ellas.

1.3. Asignación de valores con SET

Una vez declarada una variable, podemos asignarle un valor mediante la instrucción `SET`.

Sintaxis:

```
SET nombre_variable = valor;
```

Ejemplo: asignación de valores

```
DELIMITER //

CREATE PROCEDURE ejemplo_asignacion()
BEGIN
    DECLARE cantidad INT;
    DECLARE producto VARCHAR(100);
    DECLARE precio DECIMAL(10,2);

    SET cantidad = 10;
    SET producto = 'Teclado';
    SET precio = 25.50;

    SELECT cantidad, producto, precio;
END //

DELIMITER ;
```

Las variables almacenan los valores asignados y posteriormente se muestran mediante una consulta SELECT.

Para ejecutar el procedimiento:

```
CALL ejemplo_asignacion();
```

1.4. Variables y consultas SELECT

Las variables también pueden utilizarse para almacenar el resultado de una consulta SQL. Para ello se utiliza la sintaxis `SELECT ... INTO`.

Esta instrucción permite recuperar un valor de una consulta y guardarlo directamente en una variable.

Ejemplo: almacenar el número de alumnos

Supongamos que tenemos la siguiente tabla:

```
CREATE TABLE alumnos (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100),
    curso VARCHAR(20)
);

INSERT INTO alumnos (nombre, curso)
VALUES
('Ana', 'DAW'),
('Luis', 'DAW'),
('Marta', 'DAM'),
('Pedro', 'DAW');
```

Podemos crear un procedimiento que almacene el número de alumnos:

```
DELIMITER //

CREATE PROCEDURE contar_alumnos()
BEGIN
    DECLARE total INT;

    SELECT COUNT(*)
    INTO total
    FROM alumnos;

    SELECT total AS numero_alumnos;
END //

DELIMITER ;
```

Al ejecutar `CALL contar_alumnos();`, la variable `total` almacenará el número de registros de la tabla.

1.5. Ámbito y duración de las variables

Las variables locales declaradas mediante `DECLARE` tienen un ámbito limitado al bloque en el que se declaran. Cuando termina la ejecución del procedimiento, dejan de estar disponibles.

Es importante diferenciarlas de las variables de usuario de MariaDB, que pueden utilizarse durante una sesión de conexión.

Característica	Variable local	Variable de usuario
Declaración	DECLARE	Se asigna directamente.
Sintaxis	nombre_variable	@nombre_variable
Ámbito	Bloque del procedimiento.	Sesión de conexión.
Duración	Durante la ejecución del bloque.	Hasta que termina la sesión.

Ejemplo: variable de usuario

```
SET @nombre = 'Ana';

SET @edad = 25;

SELECT @nombre, @edad;
```

Las variables precedidas por `@` pueden utilizarse directamente desde una sesión de MariaDB sin necesidad de crear un procedimiento.

Importante:
No deben confundirse las variables locales declaradas con DECLARE y las variables de usuario precedidas por @. Las primeras se utilizan principalmente dentro de programas almacenados, mientras que las segundas pertenecen a la sesión de conexión.

2. Procedimientos almacenados

2.1. ¿Qué es un procedimiento almacenado?

Un procedimiento almacenado es un conjunto de instrucciones SQL que se guarda en el servidor de bases de datos y puede ejecutarse posteriormente mediante una llamada.

Permite agrupar diferentes operaciones en una única unidad lógica. De esta forma, una aplicación o un administrador puede ejecutar varias instrucciones SQL utilizando una sola llamada.

Los procedimientos almacenados son especialmente útiles para automatizar operaciones administrativas y reutilizar código.

2.2. Ventajas de los procedimientos almacenados

- **Reutilización:** un mismo procedimiento puede ejecutarse tantas veces como sea necesario.
- **Centralización:** la lógica de determinadas operaciones se almacena en el servidor.
- **Organización:** permite agrupar instrucciones relacionadas.
- **Seguridad:** pueden concederse permisos de ejecución sin necesidad de proporcionar acceso directo a todas las tablas.
- **Mantenimiento:** permite modificar la lógica de una operación en un único lugar.

2.3. Creación de procedimientos con CREATE PROCEDURE

Para crear un procedimiento almacenado se utiliza la instrucción `CREATE PROCEDURE`.

Sintaxis general:

```
DELIMITER //

CREATE PROCEDURE nombre_procedimiento()
BEGIN
    -- Instrucciones SQL
END //

DELIMITER ;
```

En esta estructura aparecen varios elementos:

- `DELIMITER //` cambia temporalmente el delimitador de instrucciones.
- `CREATE PROCEDURE` crea el procedimiento.
- `BEGIN` indica el comienzo del bloque de instrucciones.
- Las instrucciones SQL contienen las operaciones que se ejecutarán.
- `END` marca el final del procedimiento.
- `DELIMITER ;` recupera el delimitador habitual.

2.4. ¿Por qué se utiliza DELIMITER?

Normalmente, MariaDB utiliza el punto y coma (`;`) para identificar el final de una instrucción SQL.

Sin embargo, un procedimiento puede contener muchas instrucciones, cada una terminada en punto y coma. Si se utilizara el delimitador habitual durante su creación, el cliente podría interpretar que el procedimiento termina en la primera instrucción.

Para evitarlo, se cambia temporalmente el delimitador por otro símbolo, como `//` .

Ejemplo de creación de un procedimiento

```
DELIMITER //

CREATE PROCEDURE mostrar_alumnos()
BEGIN
    SELECT * FROM alumnos;
END //

DELIMITER ;
```

Durante la creación, el punto y coma termina las instrucciones internas, mientras que `//` indica el final de la definición completa del procedimiento.

Importante:
DELIMITER es una instrucción del cliente de MariaDB, no una instrucción SQL que se almacene dentro del procedimiento. Se utiliza para facilitar la introducción de bloques con múltiples instrucciones desde herramientas como la consola y Workbench.

2.5. Ejecución de procedimientos con CALL

Una vez creado el procedimiento, se puede ejecutar mediante la instrucción `CALL` .

```
CALL mostrar_alumnos();
```

No es necesario volver a escribir todas las instrucciones que contiene el procedimiento. MariaDB ejecuta directamente el código almacenado.

2.6. Modificación y eliminación de procedimientos

Para modificar un procedimiento almacenado, normalmente se elimina su definición anterior y se crea de nuevo.

```
DROP PROCEDURE IF EXISTS mostrar_alumnos;
```

Después se puede ejecutar nuevamente la instrucción `CREATE PROCEDURE` con el código actualizado.

Para consultar los procedimientos almacenados:

```
SHOW PROCEDURE STATUS;

SHOW CREATE PROCEDURE mostrar_alumnos;
```

Estas instrucciones permiten consultar los procedimientos existentes y visualizar su definición.

3. Parámetros de los procedimientos almacenados

3.1. ¿Qué son los parámetros?

Los parámetros permiten enviar información a un procedimiento cuando se ejecuta y recuperar resultados después de su ejecución.

Gracias a ellos, un mismo procedimiento puede realizar operaciones diferentes dependiendo de los valores recibidos.

Por ejemplo, se puede crear un procedimiento que consulte los alumnos de un curso determinado. El curso se enviará como parámetro.

3.2. Parámetros de entrada: IN

Los parámetros `IN` permiten enviar valores al procedimiento desde la instrucción `CALL`.

Son los parámetros más habituales y se utilizan cuando el procedimiento necesita recibir datos para realizar una operación.

Ejemplo: consultar alumnos de un curso

```
DELIMITER //

CREATE PROCEDURE alumnos_por_curso(
    IN p_curso VARCHAR(20)
)
BEGIN
    SELECT *
    FROM alumnos
    WHERE curso = p_curso;
END //

DELIMITER ;
```

Para ejecutar el procedimiento:

```
CALL alumnos_por_curso('DAW');
```

El valor DAW se envía al parámetro de entrada `p_curso`. El procedimiento devuelve únicamente los alumnos que pertenecen a ese curso.

3.3. Parámetros de salida: OUT

Los parámetros `OUT` permiten devolver información desde el procedimiento hacia el programa o la sesión que lo ejecuta.

El procedimiento asigna un valor al parámetro y el usuario puede recuperarlo utilizando una variable de sesión.

Ejemplo: devolver el número de alumnos

```
DELIMITER //

CREATE PROCEDURE total_alumnos(
    OUT p_total INT
)
BEGIN
    SELECT COUNT(*)
    INTO p_total
    FROM alumnos;
END //

DELIMITER ;
```

Para ejecutar el procedimiento y recuperar el resultado:

```
CALL total_alumnos(@resultado);

SELECT @resultado AS total_alumnos;
```

El valor calculado se almacena en la variable de sesión `@resultado`.

3.4. Parámetros de entrada y salida: INOUT

Los parámetros `INOUT` permiten recibir un valor inicial y devolver otro valor después de ejecutar el procedimiento.

Se utilizan cuando el procedimiento necesita modificar un valor que recibe como entrada.

Ejemplo: incrementar una cantidad

```
DELIMITER //

CREATE PROCEDURE incrementar(
    INOUT p_numero INT
)
BEGIN
    SET p_numero = p_numero + 1;
END //

DELIMITER ;
```

Ejecución:

```
SET @valor = 10;

CALL incrementar(@valor);

SELECT @valor;
```

La variable comienza con el valor 10 y después de ejecutar el procedimiento contiene el valor 11.

3.5. Comparación de los tipos de parámetros

Tipo	Función	Ejemplo de uso
IN	Recibe un valor desde el exterior.	Consultar alumnos de un curso.
OUT	Devuelve un resultado.	Devolver el número de alumnos.
INOUT	Recibe y devuelve un valor.	Incrementar una cantidad recibida.

4. Estructuras de control de flujo

4.1. ¿Qué son las estructuras de control?

Las estructuras de control permiten modificar el orden habitual de ejecución de las instrucciones de un programa.

Sin estructuras de control, las instrucciones de un procedimiento se ejecutarían siempre de forma secuencial, una detrás de otra.

Mediante las estructuras condicionales y repetitivas es posible tomar decisiones y ejecutar determinadas instrucciones varias veces.

Se distinguen dos grandes grupos:

- **Estructuras condicionales:** permiten ejecutar instrucciones dependiendo del resultado de una condición.
- **Estructuras repetitivas:** permiten ejecutar un conjunto de instrucciones varias veces.

5. Estructuras condicionales

5.1. Estructura IF

La estructura `IF` permite ejecutar un bloque de instrucciones cuando se cumple una condición.

Sintaxis:

```
IF condicion THEN
    instrucciones;
END IF;
```

Ejemplo: comprobar una cantidad

```
DELIMITER //

CREATE PROCEDURE comprobar_cantidad(
    IN p_cantidad INT
)
BEGIN
    IF p_cantidad > 10 THEN
        SELECT 'Cantidad elevada' AS mensaje;
    END IF;
END //

DELIMITER ;
```

Si el valor recibido es superior a 10, se muestra el mensaje. En caso contrario, no se ejecuta ninguna instrucción dentro del bloque IF.

```
CALL comprobar_cantidad(15);
```

5.2. Estructura IF ... ELSE

Cuando necesitamos ejecutar una alternativa si la condición no se cumple, podemos utilizar `ELSE`.

Sintaxis:

```
IF condicion THEN
    instrucciones_si_verdadero;
ELSE
    instrucciones_si_falso;
END IF;
```

Ejemplo: comprobar si un alumno está matriculado

```
DELIMITER //

CREATE PROCEDURE comprobar_alumno(
    IN p_id INT
)
BEGIN
    DECLARE total INT;

    SELECT COUNT(*)
    INTO total
    FROM alumnos
    WHERE id = p_id;

    IF total > 0 THEN
        SELECT 'El alumno existe' AS mensaje;
    ELSE
        SELECT 'El alumno no existe' AS mensaje;
    END IF;
END //

DELIMITER ;
```

El procedimiento comprueba si existe un alumno con el identificador recibido y muestra un mensaje diferente dependiendo del resultado.

5.3. Estructura IF ... ELSEIF ... ELSE

Cuando existen varias posibilidades, se pueden utilizar diferentes condiciones mediante `ELSEIF`.

Sintaxis:

```
IF condicion1 THEN
    instrucciones1;
ELSEIF condicion2 THEN
    instrucciones2;
ELSEIF condicion3 THEN
    instrucciones3;
ELSE
    instrucciones_alternativas;
END IF;
```

Ejemplo: clasificar un número de alumnos

```
DELIMITER //

CREATE PROCEDURE clasificar_grupo(
    IN p_total INT
)
BEGIN
    IF p_total = 0 THEN
        SELECT 'Grupo vacío' AS estado;

    ELSEIF p_total <= 15 THEN
        SELECT 'Grupo pequeño' AS estado;

    ELSEIF p_total <= 30 THEN
        SELECT 'Grupo normal' AS estado;

    ELSE
        SELECT 'Grupo numeroso' AS estado;
    END IF;
END //

DELIMITER ;
```

Dependiendo del número de alumnos recibido, el procedimiento clasifica el grupo en una de las cuatro categorías.

5.4. Operadores utilizados en las condiciones

Las condiciones de las estructuras IF utilizan operadores de comparación y operadores lógicos.

Operador	Significado
=	Igual.
<>	Diferente.
>	Mayor que.
<	Menor que.
>=	Mayor o igual.
<=	Menor o igual.
AND	Se deben cumplir ambas condiciones.
OR	Debe cumplirse al menos una condición.
NOT	Invierte el resultado lógico.

Ejemplo: utilizar varias condiciones

```
IF p_edad >= 18 AND p_edad <= 65 THEN
    SELECT 'Edad laboral' AS mensaje;
ELSE
    SELECT 'Fuera del intervalo' AS mensaje;
END IF;
```

6. Estructuras repetitivas

Las estructuras repetitivas permiten ejecutar varias veces un bloque de instrucciones. Son especialmente útiles cuando necesitamos procesar datos de forma iterativa o realizar operaciones administrativas repetidas.

6.1. Bucle WHILE

El bucle `WHILE` ejecuta un conjunto de instrucciones mientras se cumpla una condición.

La condición se comprueba antes de cada repetición. Si inicialmente no se cumple, el bloque no se ejecuta ninguna vez.

Sintaxis:

```
WHILE condicion DO
    instrucciones;
END WHILE;
```

Ejemplo: mostrar los números del 1 al 5

```
DELIMITER //

CREATE PROCEDURE numeros_while()
BEGIN
    DECLARE contador INT DEFAULT 1;

    WHILE contador <= 5 DO

        SELECT contador;

        SET contador = contador + 1;

    END WHILE;
END //

DELIMITER ;
```

El procedimiento comienza con el valor 1. En cada repetición muestra el valor actual e incrementa el contador. Cuando llega a 6, la condición deja de cumplirse.

6.2. Bucle REPEAT

El bucle `REPEAT` ejecuta un conjunto de instrucciones hasta que se cumple una condición.

A diferencia de WHILE, la condición se comprueba al final del bloque. Por ello, el contenido se ejecuta al menos una vez.

Sintaxis:

```
REPEAT
    instrucciones;
UNTIL condicion
END REPEAT;
```

Ejemplo: contar hasta 5 con REPEAT

```
DELIMITER //

CREATE PROCEDURE numeros_repeat()
BEGIN
    DECLARE contador INT DEFAULT 1;

    REPEAT

        SELECT contador;

        SET contador = contador + 1;

    UNTIL contador > 5
    END REPEAT;
END //

DELIMITER ;
```

En este caso, las instrucciones se ejecutan primero y posteriormente se comprueba si el contador ha superado 5.

6.3. Bucle LOOP

El bucle `LOOP` permite repetir instrucciones indefinidamente hasta que se ejecuta una instrucción que interrumpe el bucle.

Para abandonar el bucle se utiliza `LEAVE`, normalmente junto con una etiqueta que identifica el bloque.

Sintaxis:

```
nombre_bucle: LOOP

    instrucciones;

    IF condicion_salida THEN
        LEAVE nombre_bucle;
    END IF;

END LOOP nombre_bucle;
```

Ejemplo: contar hasta 5 con LOOP

```
DELIMITER //

CREATE PROCEDURE numeros_loop()
BEGIN
    DECLARE contador INT DEFAULT 1;

    bucle: LOOP

        SELECT contador;

        SET contador = contador + 1;

        IF contador > 5 THEN
            LEAVE bucle;
        END IF;

    END LOOP bucle;
END //

DELIMITER ;
```

El bucle continúa ejecutándose hasta que el contador supera 5. En ese momento, `LEAVE bucle` termina la repetición.

6.4. Comparación de las estructuras repetitivas

Estructura	Comprobación	Características
WHILE	Al principio.	Puede no ejecutarse ninguna vez.
REPEAT	Al final.	Se ejecuta al menos una vez.
LOOP	No tiene condición propia.	Necesita una salida explícita, por ejemplo LEAVE.

6.5. Control de iteraciones con ITERATE

Además de `LEAVE`, MariaDB dispone de `ITERATE`, que permite volver al comienzo de una iteración de un bucle.

Es similar a la instrucción CONTINUE de otros lenguajes de programación.

Ejemplo: mostrar solamente números pares

```
DELIMITER //

CREATE PROCEDURE mostrar_pares()
BEGIN
    DECLARE contador INT DEFAULT 0;

    bucle: LOOP

        SET contador = contador + 1;

        IF contador > 10 THEN
            LEAVE bucle;
        END IF;

        IF MOD(contador, 2) <> 0 THEN
            ITERATE bucle;
        END IF;

        SELECT contador;

    END LOOP bucle;
END //

DELIMITER ;
```

El procedimiento recorre los números del 1 al 10. Cuando encuentra un número impar, utiliza ITERATE para continuar con la siguiente repetición sin ejecutar la consulta SELECT.

7. Automatización de operaciones administrativas

7.1. Procedimientos para operaciones DDL

Las instrucciones DDL permiten definir y modificar la estructura de las bases de datos.

Entre las operaciones DDL más habituales se encuentran:

- `CREATE DATABASE`: crear bases de datos.
- `CREATE TABLE`: crear tablas.
- `ALTER TABLE`: modificar estructuras.
- `DROP TABLE`: eliminar tablas.

- `DROP DATABASE`: eliminar bases de datos.

Estas operaciones también pueden incorporarse a procedimientos almacenados para automatizar tareas de administración.

Ejemplo: crear una tabla de registros de actividad

```
DELIMITER //

CREATE PROCEDURE crear_tabla_log()
BEGIN
    CREATE TABLE IF NOT EXISTS registro_actividad (
        id INT AUTO_INCREMENT PRIMARY KEY,
        descripcion VARCHAR(200) NOT NULL,
        fecha DATETIME DEFAULT CURRENT_TIMESTAMP
    );
END //

DELIMITER ;
```

Cada vez que se ejecuta el procedimiento se comprueba la existencia de la tabla y se crea si todavía no existe.

```
CALL crear_tabla_log();
```

7.2. Procedimientos para operaciones DML

Las instrucciones DML permiten consultar y modificar los datos almacenados en las tablas.

Entre las operaciones más habituales se encuentran:

- `SELECT`: consultar información.
- `INSERT`: insertar registros.
- `UPDATE`: modificar registros.
- `DELETE`: eliminar registros.

Ejemplo: actualizar los datos de un alumno

```
DELIMITER //

CREATE PROCEDURE cambiar_curso(
    IN p_id INT,
    IN p_nuevo_curso VARCHAR(20)
)
BEGIN
    UPDATE alumnos
    SET curso = p_nuevo_curso
    WHERE id = p_id;
END //

DELIMITER ;
```

El procedimiento recibe el identificador del alumno y el nuevo curso. Después realiza la actualización.

```
CALL cambiar_curso(1, 'DAM');
```

7.3. Ejecución de varias instrucciones

Una de las principales ventajas de los procedimientos almacenados es que permiten ejecutar varias instrucciones dentro de un mismo bloque.

Por ejemplo, un procedimiento administrativo puede:

1. Comprobar si existe un registro.
2. Insertar información en una tabla de auditoría.
3. Actualizar los datos de una tabla principal.
4. Registrar el resultado de la operación.

Ejemplo: registrar una modificación

```
CREATE TABLE IF NOT EXISTS auditoria_alumnos (
    id INT AUTO_INCREMENT PRIMARY KEY,
    alumno_id INT,
    curso_anterior VARCHAR(20),
    curso_nuevo VARCHAR(20),
    fecha DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

```
DELIMITER //

CREATE PROCEDURE actualizar_alumno(
    IN p_id INT,
    IN p_curso VARCHAR(20)
)
BEGIN
    DECLARE curso_antiguo VARCHAR(20);

    SELECT curso
    INTO curso_antiguo
    FROM alumnos
    WHERE id = p_id;

    UPDATE alumnos
    SET curso = p_curso
    WHERE id = p_id;

    INSERT INTO auditoria_alumnos (
        alumno_id,
        curso_anterior,
        curso_nuevo
    )
    VALUES (
        p_id,
        curso_antiguo,
        p_curso
    );
END //

DELIMITER ;
```

Este procedimiento actualiza el curso de un alumno y registra el valor anterior y el nuevo en una tabla de auditoría.

> **Consideración importante:**
> El ejemplo anterior presupone que el identificador del alumno existe. En un procedimiento real es necesario comprobarlo antes de continuar. También es recomendable utilizar transacciones para asegurar que la actualización y el registro de auditoría se realizan conjuntamente.

8. Tratamiento básico de errores

8.1. ¿Por qué es necesario controlar los errores?

Durante la ejecución de un procedimiento pueden producirse diferentes errores:

- Intentar insertar un valor duplicado en una clave primaria.
- Introducir un valor que incumple una restricción.
- Consultar una tabla inexistente.
- Realizar una operación sobre un registro que no existe.
- Producir un error de integridad referencial.

Si no se controlan adecuadamente, estos errores pueden interrumpir la ejecución del procedimiento y dejar operaciones incompletas.

8.2. Manejadores de errores: DECLARE HANDLER

MariaDB permite definir manejadores de condiciones mediante `DECLARE HANDLER`.

Estos manejadores especifican qué debe hacer el procedimiento cuando se produce una determinada condición.

Sintaxis general:

```
DECLARE tipo_manejador HANDLER
FOR condicion
instruccion;
```

Los tipos de manejadores principales son:

Manejador	Comportamiento
CONTINUE	Ejecuta el manejador y continúa con la ejecución.
EXIT	Ejecuta el manejador y termina el bloque BEGIN ... END donde está declarado.

Algunas condiciones habituales son:

- `NOT FOUND`: no se ha encontrado un registro o resultado.
- `SQLEXCEPTION`: se ha producido una excepción SQL.
- `SQLWARNING`: se ha producido una advertencia SQL.

Ejemplo: controlar una excepción SQL

```
DELIMITER //

CREATE PROCEDURE insertar_alumno(
    IN p_id INT,
    IN p_nombre VARCHAR(100),
    IN p_curso VARCHAR(20)
)
BEGIN

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        SELECT 'Se ha producido un error al insertar el alumno'
        AS mensaje;
    END;

    INSERT INTO alumnos (id, nombre, curso)
    VALUES (p_id, p_nombre, p_curso);

    SELECT 'Alumno insertado correctamente' AS mensaje;

END //

DELIMITER ;
```

Si se produce una excepción SQL durante la inserción, se ejecuta el manejador EXIT y se muestra un mensaje. El procedimiento termina después de ejecutar el manejador.

8.3. Capturar el código y el mensaje del error

MariaDB proporciona funciones para obtener información sobre la condición que ha provocado un error.

- `MYSQL_ERRNO`: devuelve el código numérico del error.
- `MESSAGE_TEXT`: devuelve el mensaje descriptivo.
- `RETURNED_SQLSTATE`: devuelve el código SQLSTATE.

Se pueden utilizar dentro de un manejador mediante `GET DIAGNOSTICS`.

Ejemplo: obtener información del error

```
DELIMITER //

CREATE PROCEDURE insertar_controlado(
    IN p_id INT,
    IN p_nombre VARCHAR(100),
    IN p_curso VARCHAR(20)
)
BEGIN

    DECLARE v_codigo INT;
    DECLARE v_mensaje TEXT;

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        GET DIAGNOSTICS CONDITION 1
            v_codigo = MYSQL_ERRNO,
            v_mensaje = MESSAGE_TEXT;

        SELECT v_codigo AS codigo_error,
               v_mensaje AS descripcion_error;
    END;

    INSERT INTO alumnos (id, nombre, curso)
    VALUES (p_id, p_nombre, p_curso);

    SELECT 'Operación realizada correctamente' AS resultado;

END //

DELIMITER ;
```

Cuando se produce un error SQL, el manejador obtiene su código y descripción para mostrar información más detallada.

Importante:
Capturar una excepción no significa que los datos se hayan recuperado automáticamente. Si una operación ha modificado información antes de producirse el error, puede ser necesario utilizar transacciones y ejecutar ROLLBACK para evitar inconsistencias.

9. Integridad y consistencia en los procedimientos

9.1. Riesgos de las operaciones automatizadas

Un procedimiento almacenado puede ejecutar múltiples operaciones sobre diferentes tablas. Por ello, es importante garantizar que los cambios realizados mantienen la integridad de la información.

Por ejemplo, si un procedimiento transfiere un alumno de un curso a otro y registra el movimiento en una tabla de historial, ambas operaciones deberían completarse correctamente.

9.2. Uso de transacciones

Las transacciones permiten agrupar varias operaciones para que se confirmen conjuntamente o se deshagan si se produce un problema.

Las instrucciones principales son:

- `START TRANSACTION`: inicia una transacción.
- `COMMIT`: confirma los cambios.
- `ROLLBACK`: deshace los cambios pendientes.

Ejemplo: operación con transacción y control de errores

```
DELIMITER //

CREATE PROCEDURE actualizar_y_auditar(
    IN p_id INT,
    IN p_curso VARCHAR(20)
)
BEGIN

    DECLARE v_curso_antiguo VARCHAR(20);

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SELECT 'Error: operación cancelada' AS resultado;
    END;

    START TRANSACTION;

    SELECT curso
    INTO v_curso_antiguo
    FROM alumnos
    WHERE id = p_id
    FOR UPDATE;

    UPDATE alumnos
    SET curso = p_curso
    WHERE id = p_id;

    INSERT INTO auditoria_alumnos (
        alumno_id,
        curso_anterior,
        curso_nuevo
    )
    VALUES (
        p_id,
        v_curso_antiguo,
        p_curso
    );

    COMMIT;

    SELECT 'Operación completada' AS resultado;

END //

DELIMITER ;
```

El procedimiento inicia una transacción, consulta el curso actual, actualiza el alumno y registra el cambio en la tabla de auditoría.

Si se produce una excepción SQL, el manejador ejecuta `ROLLBACK` para deshacer las operaciones pendientes. Si todo funciona correctamente, se ejecuta `COMMIT`.

Requisito del motor de almacenamiento
Para que las transacciones permitan deshacer los cambios correctamente, las tablas implicadas deben utilizar un motor transaccional, como InnoDB. Las operaciones DDL, como CREATE TABLE o DROP TABLE, pueden provocar confirmaciones implícitas y no deben tratarse como operaciones transaccionales ordinarias.

10. Ejemplo integrador: automatización de la gestión de alumnos

En este ejemplo se combinan variables, parámetros, estructuras condicionales, operaciones DML y control de errores para automatizar una operación administrativa.

10.1. Estructura de la base de datos

```
CREATE DATABASE IF NOT EXISTS centro;
USE centro;

CREATE TABLE IF NOT EXISTS alumnos (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    curso VARCHAR(20) NOT NULL,
    activo BOOLEAN DEFAULT TRUE
) ENGINE=InnoDB;
```

10.2. Procedimiento para cambiar un alumno de curso

```
DELIMITER //

CREATE PROCEDURE cambiar_curso_alumno(
    IN p_id INT,
    IN p_nuevo_curso VARCHAR(20),
    OUT p_resultado VARCHAR(100)
)
BEGIN

    DECLARE v_total INT;

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SET p_resultado = 'Error durante la operación';
    END;

    START TRANSACTION;

    SELECT COUNT(*)
    INTO v_total
    FROM alumnos
    WHERE id = p_id
      AND activo = TRUE;

    IF v_total = 0 THEN

        SET p_resultado = 'El alumno no existe o está inactivo';

        ROLLBACK;

    ELSE

        UPDATE alumnos
        SET curso = p_nuevo_curso
        WHERE id = p_id;

        COMMIT;

        SET p_resultado = 'Curso actualizado correctamente';

    END IF;

END //

DELIMITER ;
```

10.3. Ejecución del procedimiento

```
CALL cambiar_curso_alumno(
    1,
    'DAM',
    @mensaje
);

SELECT @mensaje AS resultado;
```

El procedimiento realiza las siguientes operaciones:

1. Recibe el identificador del alumno y el nuevo curso.
2. Comprueba si existe un alumno activo con ese identificador.
3. Si no existe, cancela la operación y devuelve un mensaje.
4. Si existe, actualiza su curso.
5. Confirma la transacción.
6. Devuelve el resultado mediante un parámetro OUT.

> **Conceptos utilizados en el ejemplo**
>
> - Variables locales mediante DECLARE.
> - Asignación de valores mediante SET.
> - Parámetros IN y OUT.
> - Consulta SELECT INTO.
> - Condicional IF ... ELSE.
> - Operación DML UPDATE.
> - Control de errores mediante DECLARE HANDLER.
> - Transacciones mediante COMMIT y ROLLBACK.

11. Resumen de conceptos fundamentales

Concepto	Descripción
DECLARE	Declara variables locales dentro de un bloque.
SET	Asigna valores a las variables.
SELECT INTO	Almacena el resultado de una consulta en variables.
CREATE PROCEDURE	Crea un procedimiento almacenado.
CALL	Ejecuta un procedimiento almacenado.
IN	Parámetro de entrada.
OUT	Parámetro de salida.
INOUT	Parámetro de entrada y salida.
IF / ELSEIF / ELSE	Permiten ejecutar instrucciones dependiendo de condiciones.
WHILE	Repite instrucciones mientras se cumple una condición.
REPEAT	Repite instrucciones hasta que se cumple una condición.
LOOP	Permite crear un bucle con salida explícita.
LEAVE	Abandona un bloque etiquetado.
ITERATE	Salta al comienzo de la siguiente iteración.
DECLARE HANDLER	Define cómo actuar ante determinadas condiciones o errores.
COMMIT	Confirma una transacción.
ROLLBACK	Deshace los cambios pendientes de una transacción.

12. Ejercicios prácticos de aprendizaje

Los siguientes ejercicios permiten practicar la creación de procedimientos almacenados y el uso de estructuras de control en MariaDB.

Ejercicio 1. Variables y procedimientos

Trabajar con la base de datos `centro` y su tabla `alumnos`.

1. Crear un procedimiento que declare una variable entera.
2. Asignarle un valor mediante SET.
3. Mostrar el contenido de la variable mediante SELECT.
4. Crear otro procedimiento que almacene en una variable el número de alumnos de la tabla.
5. Mostrar el resultado de la consulta.

Ejercicio 2. Parámetros de entrada y salida

1. Crear un procedimiento que reciba el identificador de un alumno mediante IN.
2. Consultar sus datos y devolverlos mediante una consulta SELECT.
3. Crear otro procedimiento que reciba un curso y devuelva el número de alumnos mediante OUT.
4. Crear un procedimiento que utilice INOUT para incrementar una cantidad recibida.
5. Probar los procedimientos con diferentes valores.

Ejercicio 3. Estructuras condicionales

1. Crear un procedimiento que reciba una cantidad de alumnos.
2. Utilizar IF para comprobar si el grupo está vacío.
3. Utilizar ELSEIF para clasificar los grupos según su tamaño.
4. Utilizar ELSE para gestionar los valores que no encajen en las condiciones anteriores.
5. Ejecutar el procedimiento con diferentes cantidades.

Ejercicio 4. Estructuras repetitivas

1. Crear un procedimiento que muestre los números del 1 al 10 mediante WHILE.
2. Repetir la operación utilizando REPEAT.
3. Repetir la operación utilizando LOOP y LEAVE.
4. Crear un procedimiento que muestre únicamente los números pares entre 1 y 20 utilizando ITERATE.
5. Comparar el funcionamiento de las tres estructuras.

Ejercicio 5. Automatización de operaciones administrativas

1. Crear una tabla para registrar operaciones administrativas.
2. Crear un procedimiento que permita registrar una nueva operación.
3. Utilizar parámetros de entrada para recibir la descripción de la operación.
4. Incluir una variable que almacene la fecha actual.
5. Crear una consulta para mostrar el historial de operaciones.

Ejercicio 6. Control de errores

1. Crear un procedimiento que inserte un nuevo alumno.
2. Definir un manejador EXIT para controlar excepciones SQL.
3. Recuperar el código y el mensaje del error mediante GET DIAGNOSTICS.
4. Probar la inserción de un identificador duplicado.
5. Comprobar el resultado del procedimiento.

Ejercicio 7. Integridad mediante transacciones

1. Crear una tabla de historial de cambios de curso.
2. Crear un procedimiento que actualice el curso de un alumno.
3. Registrar el cambio en la tabla de historial.
4. Utilizar START TRANSACTION y COMMIT.
5. Incorporar un manejador de errores que ejecute ROLLBACK.
6. Comprobar que una operación fallida no deja cambios parciales.

> **Resultado esperado**
> Al finalizar este bloque, el alumnado debe ser capaz de desarrollar procedimientos almacenados que utilicen variables, parámetros, estructuras condicionales y repetitivas. También debe poder automatizar operaciones administrativas y aplicar mecanismos básicos de control de errores e integridad de la información.

---

13. Relación con los criterios de evaluación

Criterio	Aprendizaje esperado
CA4.4	Definir y utilizar procedimientos y secuencias de comandos para automatizar tareas administrativas.
CA4.7	Utilizar estructuras condicionales y repetitivas para controlar la ejecución de los procedimientos.
CA4.8	Aplicar mecanismos de control de errores y transacciones para mantener la integridad y consistencia de los datos.